Web Development using PHP

Object Oriented Programming

(Lecture 6)

Prepared By: *BITS IT Team*

Venue: *GS Training Room*

Table of Contents

1. Defining Class Properties 4
Example 1.1 4
Example 1.2 4
2. Defining Class Methods 5
Example 2.1 5
Example 2.2 6
Example 2.3 6
3. Using Constructors and Destructors 8
Example 3.1 8
Example 3.2 9
4. Using Class Inheritance 10
Example 4.1 10
4.1 Overwriting Inherited Properties and Methods 11
Example 4.2 11
4.2 Preserving Original Method Functionality While Overwriting Methods 12
Example 4.3 12
5. Assigning the Visibility of Properties and Methods 13
5.1 Public Properties and Methods 13
5.2 Protected Properties and Methods 13
Example 5.1 13
Example 5.2 15
5.3 Private Properties and Methods 16
Example 5.3 16
5.4 Static Properties and Methods 17
Example 5.4 17
6 Resources 20
6.1 websites 20
6.2 Books 20

Continuing with our last lecture, we'll now cover other details of our last program which was;

```
<html>
<body>
<?php
class my_lectures
{
   // Class properties and methods go here
}

$obj = new my_lectures;
var_dump($obj);
?>
</body>
</html>
```

1. Defining Class Properties

To add data to a class, properties, or class-specific variables, are used. These work exactly like regular variables, except they're bound to the object and therefore can only be accessed using the object.

To add a property to my_lectures, add the following code to your script.

Example 1.1

```
<html>
<body>
<?php
class my_lectures
{
   // Class properties and methods go here
   public $prop1 = "I'm a class property!";
}

$obj = new my_lectures;
var_dump($obj);
?>
</body>
</html>
```

The keyword public determines the visibility of the property, which we'll learn about a little later. Next, the property is named using standard variable syntax, and a value is assigned (though class properties do not need an initial value).

To read this property and output it to the browser, reference the object from which to read and the property to be read.

Example 1.2

```
<html>
<body>
<?php
```

```
class my_lectures
{
   // Class properties and methods go here
   public $prop1 = "I'm a class property!";
}

$obj = new my_lectures;
// var_dump($obj);
echo $obj->prop1; // Output the property
?>
</body>
</html>
```

Reloading your browser now outputs the following;

```
I'm a class property!
```

Run the page in the browser a button having label "Click Me to get a Message!" should display.

2. Defining Class Methods

Methods are class-specific functions. Individual actions that an object will be able to perform are defined within the class as methods.

For instance, to create methods that would set and get the value of the class property $prop1, add the following to your code;

Example 2.1

```
<html>
<body>
<?php
class my_lectures
{
   // Class properties and methods go here
   public $prop1 = "I'm a class property!";

   public function set_property($newval)
   {
      $this->prop1 = $newval;
   }

   public function get_property()
   {
      return $this->prop1 . "<br>";
   }
}

$obj = new my_lectures;

echo $obj->prop1; // Output the property
?>
```

```
</body>
</html>
```

Note — *OOP allows objects to reference themselves using* ***$this***. When working within a method, use ***$this*** in the same way you would use the object name outside the class.

To use these methods, call them just like regular functions, but first, reference the object they belong to. Read the property from my_lectures, change its value, and read it out again by making the modifications as follows;

Example 2.2

```
<html>
<body>
<?php
class my_lectures
{
   // Class properties and methods go here
   public $prop1 = "I'm a class property!";

   public function set_property($newval)
   {
      $this->prop1 = $newval;
   }

   public function get_property()
   {
      return $this->prop1 . "<br>";
   }
}

$obj = new my_lectures;

echo $obj->get_property(); // Get the property value

$obj->set_property("I'm a new property value!"); // Set a new one

echo $obj->get_property(); // Read it again to show the change
?>
</body>
</html>
```

Reload your browser; you should see the following;

```
I'm a class property!

I'm a new property value!
```

Now extend our example and use two different objects as follows;

Example 2.3

```
<html>
<body>
<?php
```

```
class my_lectures
{
   // Class properties and methods go here
   public $prop1 = "I'm a class property!";

   public function set_property($newval)
   {
      $this->prop1 = $newval;
   }

   public function get_property()
   {
      return $this->prop1 . "<br>";
   }
}

// Create two objects
$obj = new my_lectures;
$obj2 = new my_lectures;

// Get the value of $prop1 from both objects
echo $obj->get_property();
echo $obj2->get_property();

// Set new values for both objects
$obj->set_property("I'm a new property value!");
$obj2->set_property("I belong to the second instance!");

// Output both objects' $prop1 value
echo $obj->get_property();
echo $obj2->get_property();
?>
</body>
</html>
```

Run the page in the browser. Following output should be read.

```
I'm a class property!
I'm a class property!
I'm a new property value!
I belong to the second instance!
```

To make the use of objects easier, PHP also provides some special methods that are called when certain common actions occur within objects. This allows developers to perform a number of useful tasks with relative ease. You must be familiar with these; these are ever known as **Constructors** and **Destructors**.

3. Using Constructors and Destructors

When an object is instantiated, it's often desirable to set a few things right after the program execute. To handle this, PHP provides the special method **__construct(),** which is called automatically whenever a new object is created.

To illustrate this we'll add a constructor to my_lectures that will output a message whenever a new instance of the class is created;

Example 3.1

```
<html>
<body>
<?php
class my_lectures
{
   // Class properties and methods go here
   public $prop1 = "I'm a class property!";
   public function __construct()
   {
      echo 'The class "', __CLASS__, '" was initiated!<br />';
   }
   public function set_property($newval)
   {
      $this->prop1 = $newval;
   }

   public function get_property()
   {
      return $this->prop1 . "<br>";
   }
}

// Create two objects
$obj = new my_lectures;

// Get the value of $prop1 from both objects
echo $obj->get_property();

// Output a message at the end of the file
echo "End of file.<br />";
?>
</body>
</html>
```

Run the page in the browser. Following output should be read.

```
The class "my_lectures" was initiated!
I'm a class property!
End of file.
```

Note: __CLASS__ returns the name of the class in which it is called.

To call a function when the object is destroyed, the __destruct() special method is available. This is useful for class cleanup (closing a database connection, for instance).

Following example Outputs a message when the object is destroyed.

Example 3.2

```
<html>
<body>
<?php
class my_lectures
{
   // Class properties and methods go here
   public $prop1 = "I'm a class property!";
   public function __construct()
   {
      echo 'The class "', __CLASS__, '" was initiated!<br />';
   }

   public function __destruct()
   {
      echo 'The class "', __CLASS__, '" was destroyed.<br />';
   }
   public function set_property($newval)
   {
      $this->prop1 = $newval;
   }

   public function get_property()
   {
      return $this->prop1 . "<br>";
   }
}

// Create two objects
$obj = new my_lectures;

// Get the value of $prop1 from both objects
echo $obj->get_property();

// Output a message at the end of the file
echo "End of file.<br />";
?>
</body>
</html>
```

Run the page in the browser. Following output should be read.

```
The class "my_lectures" was initiated!
I'm a class property!
End of file.
The class "my_lectures" was destroyed.
```

To explicitly trigger the destructor, you can destroy the object using the function unset(). Hence we can use unset($obj); in above example.

4. Using Class Inheritance

Classes can inherit the methods and properties of another class using the **extends** keyword. In the following example we'll create a second class that extends my_lectures and adds a method;

Example 4.1

```
<html>
<body>
<?php
class my_lectures
{
   // Class properties and methods go here
   public $prop1 = "I'm a class property!";
   public function __construct()
   {
      echo 'The class "', __CLASS__, '" was initiated!<br />';
   }

   public function __destruct()
   {
      echo 'The class "', __CLASS__, '" was destroyed.<br />';
   }
   public function set_property($newval)
   {
      $this->prop1 = $newval;
   }

   public function get_property()
   {
      return $this->prop1 . "<br>";
   }
}

class last_lecture extends my_lectures
{
   public function new_method()
   {
      echo "From a new method in " . __CLASS__ . ".<br />";
   }
}

// Create a new object
$newobj = new last_lecture;

// Output the object as a string
echo $newobj->new_method();

// Use a method from the parent class
echo $newobj->get_property();
?>
```

```
</body>
</html>
```

After running in the browser following should be the output.

```
The class "my_lectures" was initiated!
From a new method in last_lecture.
I'm a class property!
The class "my_lectures" was destroyed.
```

4.1 Overwriting Inherited Properties and Methods

To change the behavior of an existing property or method in the new class, you can simply overwrite it by **declaring it again in the new class**. Consider the following example;

Example 4.2

```
<html>
<body>
<?php
class my_lectures
{
   // Class properties and methods go here
   public $prop1 = "I'm a class property!";
   public function __construct()
   {
      echo 'The class "', __CLASS__, '" was initiated!<br />';
   }

   public function __destruct()
   {
      echo 'The class "', __CLASS__, '" was destroyed.<br />';
   }
   public function set_property($newval)
   {
      $this->prop1 = $newval;
   }

   public function get_property()
   {
      return $this->prop1 . "<br>";
   }
}

class last_lecture extends my_lectures
{
   public function __construct()
   {
      echo "A new constructor in " . __CLASS__ . ".<br />";
   }
   public function new_method()
   {
      echo "From a new method in " . __CLASS__ . ".<br />";
   }
}
```

```
// Create a new object
$newobj = new last_lecture;

// Output the object as a string
echo $newobj->new_method();

// Use a method from the parent class
echo $newobj->get_property();
?>
</body>
</html>
```

Run the example in your browser, you should see the following output;

```
A new constructor in last_lecture.
From a new method in last_lecture.
I'm a class property!
The class "my_lectures" was destroyed.
```

4.2 Preserving Original Method Functionality While Overwriting Methods

To add new functionality to an inherited method while keeping the original method intact, use the parent keyword with the **scope resolution operator** (::).

Example 4.3

```
<html>
<body>
<?php
class my_lectures
{
   // Class properties and methods go here
   public $prop1 = "I'm a class property!";
   public function __construct()
   {
      echo 'The class "', __CLASS__, '" was initiated!<br />';
   }

   public function __destruct()
   {
      echo 'The class "', __CLASS__, '" was destroyed.<br />';
   }
   public function set_property($newval)
   {
      $this->prop1 = $newval;
   }

   public function get_property()
   {
      return $this->prop1 . "<br>";
   }
}

class last_lecture extends my_lectures
{
   public function __construct()
```

```
  {
    parent::__construct(); // Call the parent class's constructor
    echo "A new constructor in " . __CLASS__ . ".<br />";
  }
  public function new_method()
  {
    echo "From a new method in " . __CLASS__ . ".<br />";
  }
}

// Create a new object
$newobj = new last_lecture;

// Output the object as a string
echo $newobj->new_method();

// Use a method from the parent class
echo $newobj->get_property();
?>
</body>
</html>
```

Run the example in your browser, you should see the following output;

```
The class "my_lectures" was initiated!
A new constructor in last_lecture.
From a new method in last_lecture.
I'm a class property!
The class "my_lectures" was destroyed.
```

5. Assigning the Visibility of Properties and Methods

For added control over objects, methods and properties are assigned visibility. This controls how and from where properties and methods can be accessed. There are three visibility keywords: **public**, **protected**, and **private**. In addition to its visibility, a method or property can be declared as static, which allows them to be accessed without an instantiation of the class.

5.1 Public Properties and Methods

All the methods and properties you've used so far have been public. This means that they can be accessed anywhere, both within the class and externally.

5.2 Protected Properties and Methods

When a property or method is declared protected, **it can only be accessed within the class itself or in descendant classes** (classes that extend the class containing the protected method).

Declare the get_property() method as protected in ***my_lectures*** and try to access it directly from outside the class;

Example 5.1

```
<html>
```

```
<body>
<?php
ini_set("display_errors","On");
class my_lectures
{
   // Class properties and methods go here
   public $prop1 = "I'm a class property!";
   public function __construct()
   {
      echo 'The class "', __CLASS__, '" was initiated!<br />';
   }

   public function __destruct()
   {
      echo 'The class "', __CLASS__, '" was destroyed.<br />';
   }
   public function set_property($newval)
   {
      $this->prop1 = $newval;
   }

   protected function get_property()
   {
      return $this->prop1 . "<br>";
   }
}

class last_lecture extends my_lectures
{
   public function __construct()
   {
      parent::__construct(); // Call the parent class's constructor
      echo "A new constructor in " . __CLASS__ . ".<br />";
   }
   public function new_method()
   {
      echo "From a new method in " . __CLASS__ . ".<br />";
   }
}

// Create a new object
$newobj = new last_lecture;

// Use a method from the parent class
echo $newobj->get_property();
?>
</body>
</html>
```

Run the example in your browser, you should see the following output;

```
The class "my_lectures" was initiated!
A new constructor in last_lecture.

Fatal error: Call to protected method my_lectures::get_property() from context '' in C:\Program
Files\Zend\Apache2\htdocs\training\Lecture6\example5.1.php on line 49
```

Now, create a new method in ***last_lecture*** to call the get_property() method as follows;

Example 5.2

```
<html>
<body>
<?php
ini_set("display_errors","On");
class my_lectures
{
   // Class properties and methods go here
   public $prop1 = "I'm a class property!";
   public function __construct()
   {
      echo 'The class "', __CLASS__, '" was initiated!<br />';
   }

   public function __destruct()
   {
      echo 'The class "', __CLASS__, '" was destroyed.<br />';
   }
   public function set_property($newval)
   {
      $this->prop1 = $newval;
   }

   protected function get_property()
   {
      return $this->prop1 . "<br>";
   }
}

class last_lecture extends my_lectures
{
   public function __construct()
   {
      parent::__construct(); // Call the parent class's constructor
      echo "A new constructor in " . __CLASS__ . ".<br />";
   }
   public function new_method()
   {
      echo "From a new method in " . __CLASS__ . ".<br />";
   }
   public function call_protected()
   {
      return $this->get_property();
   }
}

// Create a new object
$newobj = new last_lecture;

// Call the protected method from within a public method
echo $newobj->call_protected();
?>
</body>
</html>
```

Run the example in your browser, you should see the following output;

```
The class "my_lectures" was initiated!
A new constructor in last_lecture.
I'm a class property!
The class "my_lectures" was destroyed.
```

5.3 Private Properties and Methods

A property or method declared private is accessible **only from within the class that defines it**. This means that *even if a new class extends the class that defines a private property,* that property or method will not be available at all within the child class.

To demonstrate this, declare get_property() as private in my_lectures, and attempt to call call_protected() from last_lecture.

Example 5.3

```
<html>
<body>
<?php
ini_set("display_errors","On");
class my_lectures
{
   // Class properties and methods go here
   public $prop1 = "I'm a class property!";
   public function __construct()
   {
      echo 'The class "', __CLASS__, '" was initiated!<br />';
   }

   public function __destruct()
   {
      echo 'The class "', __CLASS__, '" was destroyed.<br />';
   }
   public function set_property($newval)
   {
      $this->prop1 = $newval;
   }

   private function get_property()
   {
      return $this->prop1 . "<br>";
   }
}

class last_lecture extends my_lectures
{
   public function __construct()
   {
      parent::__construct(); // Call the parent class's constructor
      echo "A new constructor in " . __CLASS__ . ".<br />";
   }
   public function new_method()
```

```
  {
    echo "From a new method in " . __CLASS__ . ".<br />";
  }
  public function call_protected()
  {
    return $this->get_property();
  }
}

// Create a new object
$newobj = new last_lecture;

// Call the protected method from within a public method
echo $newobj->call_protected();
?>
</body>
</html>
```

Run the example in your browser, you should see the following output;

```
The class "my_lectures" was initiated!
A new constructor in last_lecture.

Fatal error: Call to private method my_lectures::get_property() from context 'last_lecture' in C:\Program
Files\Zend\Apache2\htdocs\training\Lecture6\example5.3.php on line 45
```

5.4 Static Properties and Methods

A method or property declared static can be accessed without first instantiating the class; you simply supply the class name, scope resolution operator, and the property or method name.

To demonstrate this, add a static property called $count and a static method called plus_one() to my_lectures. Then set up a do...while loop to output the incremented value of $count as long as the value is less than 10.

Example 5.4

```
<html>
<body>
<?php
ini_set("display_errors","On");
class my_lectures
{
  // Class properties and methods go here
  public $prop1 = "I'm a class property!";
  public static $count = 0;
  public function __construct()
  {
    echo 'The class "', __CLASS__, '" was initiated!<br />';
  }

  public function __destruct()
  {
    echo 'The class "', __CLASS__, '" was destroyed.<br />';
  }
```

```
   public function set_property($newval)
   {
      $this->prop1 = $newval;
   }

   private function get_property()
   {
      return $this->prop1 . "<br>";
   }
   public static function plus_one()
   {
      return "The count is " . ++self::$count . ".<br />";
   }
}

class last_lecture extends my_lectures
{
   public function __construct()
   {
      parent::__construct(); // Call the parent class's constructor
      echo "A new constructor in " . __CLASS__ . ".<br />";
   }
   public function new_method()
   {
      echo "From a new method in " . __CLASS__ . ".<br />";
   }
   public function call_protected()
   {
      return $this->get_property();
   }

}

do
{
   // Call plus_one without instantiating my_lectures
   echo my_lectures::plus_one();
} while (my_lectures::$count < 10 );
?>
</body>
</html>
```

Run the example in your browser, you should see the following output;

```
The count is 1.
The count is 2.
The count is 3.
The count is 4.
The count is 5.
The count is 6.
The count is 7.
The count is 8.
The count is 9.
The count is 10.
```

6 Resources

6.1 websites

http://www.w3schools.com/

6.2 Books

Beginning PHP5 by Wrox word press Ltd.